

## SYNCHRONOUS COUNTER

### 1. Inputs and Outputs

#### Inputs

| Input | Width | MSB   | LSB   |
|-------|-------|-------|-------|
| clk   | 1 bit | clk   | clk   |
| reset | 1 bit | reset | reset |

#### Output

| Output | Width | MSB  | LSB  |
|--------|-------|------|------|
| q[3:0] | 4-bit | q[3] | q[0] |

#### MAIN CODE

```
module sync_counter (  
    input wire    clk,  
    input wire    reset,  
    output reg [3:0] q  
);  
  
    // JK inputs for each flip-flop  
    wire j0, k0;  
    wire j1, k1;  
    wire j2, k2;  
    wire j3, k3;  
  
    // JK excitation logic  
    assign j0 = 1'b1;  
    assign k0 = 1'b1;  
  
    assign j1 = q[0];  
    assign k1 = q[0];
```

```
assign j2 = q[1] & q[0];
```

```
assign k2 = q[1] & q[0];
```

```
assign j3 = q[2] & q[1] & q[0];
```

```
assign k3 = q[2] & q[1] & q[0];
```

```
// All flip-flops use the SAME clock
```

```
always @(posedge clk or posedge reset)
```

```
begin
```

```
    if (reset)
```

```
        begin
```

```
            q <= 4'b0000;
```

```
        end
```

```
    else
```

```
        begin
```

```
            // JK toggle operation
```

```
            if (j0 && k0)
```

```
                q[0] <= ~q[0];
```

```
            if (j1 && k1)
```

```
                q[1] <= ~q[1];
```

```
            if (j2 && k2)
```

```
                q[2] <= ~q[2];
```

```
            if (j3 && k3)
```

```
                q[3] <= ~q[3];
```

```
        end
```

```
    end
```

```
endmodule
```

## TEST BENCH CODE

```
`timescale 1ns/1ps
```

```
module sync_counter_tb;
```

```
    reg clk;
```

```
    reg reset;
```

```
    wire [3:0] q;
```

```
    sync_counter uut (
```

```
        .clk(clk),
```

```
        .reset(reset),
```

```
        .q(q)
```

```
    );
```

```
    // Clock
```

```
    always #10 clk = ~clk;
```

```
    initial
```

```
    begin
```

```
        clk = 0;
```

```
        reset = 1;
```

```
        // Hold reset
```

```
        #20;
```

```
        // Release reset
```

```
        reset = 0;
```

```
// Run counter
#400;

$finish;
end

initial
begin
    $monitor("Time=%0t | CLK=%b | RESET=%b | Q=%b",
        $time, clk, reset, q);
end

endmodule
```